

CSE221: Algorithms

Max Sum Subarray

Prepared by: A B M Muntasir Rahman [MUNR]



Inspiring Excellence

Max Sum Subarray Problem

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7



max sum subarray

- *Input:* an array $A[1..n]$ of n numbers
 - Assume that some of the numbers are **negative**, because this problem is trivial when all numbers are nonnegative
- *Output:* a nonempty subarray $A[i..j]$ having the largest sum $S[i, j] = a_i + a_{i+1} + \dots + a_j$

Max Sum Subarray Problem

3	-2	5	-1
---	----	---	----

- Consider subarrays of size 1

3

-2

5

-1

Sum	Max
3	3
-2	3
5	5
1	5

- Consider subarrays of size 2

3	-2
---	----

-2	5
----	---

5	-1
---	----

Sum Max

1	5
3	5
4	5

Max Sum Subarray Problem

3	-2	5	-1
---	----	---	----

- Consider subarrays of size 3

3	-2	5
---	----	---

-2	5	-1
----	---	----

Sum Max

6	6
2	6

- Consider subarrays of size 4

3	-2	5	-1
---	----	---	----

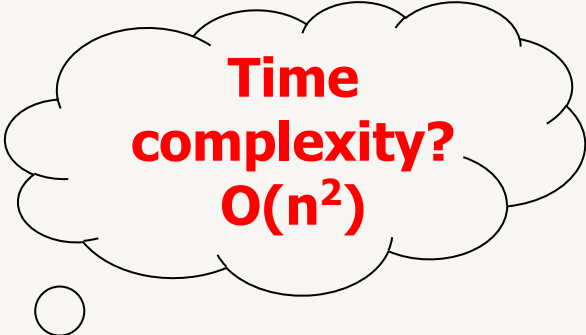
Sum Max

5	6
---	---

This is the brute force approach!

Brute Force Algorithm

```
max = -∞
for i = 1 to n do
begin
    sum = 0
    for j = i to n do
begin
        sum = sum + A[j]
        if sum > max
        then max = sum
    end
end
end
```

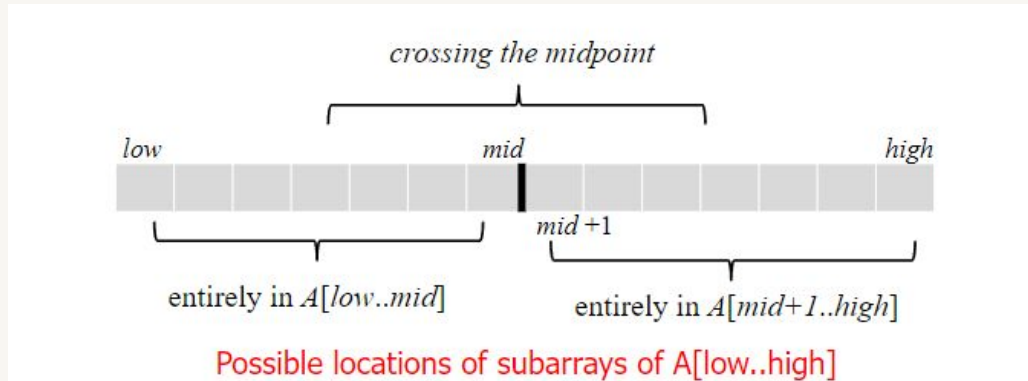


**Time
complexity?
 $O(n^2)$**

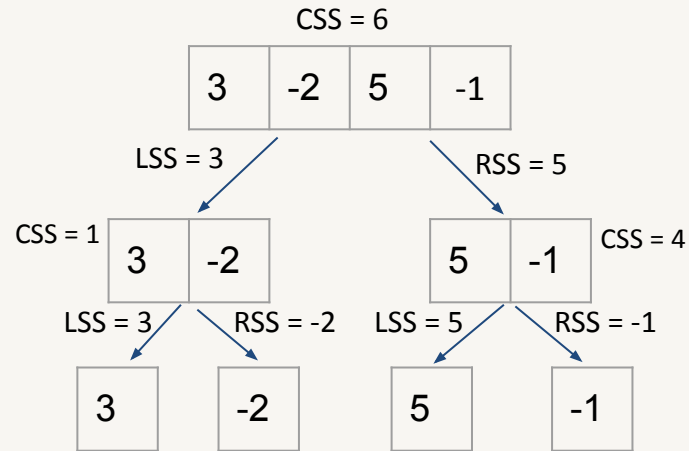
Divide & Conquer Approach

Possible locations of a maximum subarray $A[i..j]$ of $A[low..high]$, where $mid = (low + high)/2$

- entirely in $A[low..mid]$ ($low \leq i \leq j \leq mid$)
- entirely in $A[mid+1..high]$ ($mid < i \leq j \leq high$)
- crossing the midpoint ($low \leq i \leq mid < j \leq high$)



Divide & Conquer Approach



Pseudocode

```
maxSumSubArray(arr, n)
    if n == 1
        return arr[0]
    mid = n//2
    leftSum = maxSumSubArray(arr[:mid], mid)
    rightSum = maxSumSubArray(arr[mid:], n - mid)
    crossLeft, crossRight, sum = float('-inf'), float('-inf'), 0
    for i = mid - 1 to 0
        sum = sum + arr[i]
        crossLeft = max(crossLeft, sum)
    sum = 0
    for j = mid to n - 1
        sum = sum + arr[j]
        crossRight = max(crossRight, sum)
    crossSum = crossLeft + crossRight
    return max(leftSum, rightSum, crossSum)
```


Time Complexity Analysis

```
maxSumSubArray(arr, n)
```

```
    if n == 1
```

```
        return arr[0]
```

```
    mid = n//2
```

```
    leftSum = maxSumSubArray(arr[:mid], mid) → T(n/2)
```

```
    rightSum = maxSumSubArray(arr[mid:], n - mid) → T(n/2)
```

```
    crossLeft, crossRight, sum = float('-inf'), float('-inf'), 0
```

```
    for i = mid - 1 to 0 → n/2
```

```
        sum = sum + arr[i]
```

```
        crossLeft = max(crossLeft, sum)
```

```
    sum = 0
```

```
    for j = mid to n - 1 → n/2
```

```
        sum = sum + arr[j]
```

```
        crossRight = max(crossRight, sum)
```

```
    crossSum = crossLeft + crossRight
```

```
    return max(leftSum, rightSum, crossSum)
```

Time Complexity Analysis

Therefore,

$$\mathbf{T(n) = 2T(n/2) + n/2 + n/2 = 2T(n/2) + O(n)}$$

Applying Master's theorem,

$a = 2$, $b^k = 2$ and $p = 0$, therefore $a = b^k$ and $p > -1$

$$T(n) = O(n^{(\log_b a)} \log^{p+1} n) = O(n^{(\log_2 2)} \log^{0+1} n) = O(n \log n)$$